NATIONAL ECONOMICS UNIVERSITY

Faculty of Data Science and Artificial Intelligence

-------***-------

HAIR SALON MANAGEMENT DATABASE: Design and Analysis of salon_mini in MySQL

Group 1

Hoang Viet Hung	**11230749**
Ninh Thi Minh Thuy	**11230779**
Chu Viet Anh	**11230730**

Instructor: Dr. Tran Duc Minh

Hanoi, 2026

Abstract

This report presents Group 1's solution to Assignment 07 of IST2610 – Database Management: a relational database, `salon_mini`, for a hair salon that books appointments, assigns them to stylists, records the services performed and collects payments, sometimes in several instalments. We derive eleven business rules from the brief and map each to a schema constraint or to a verification query. The schema has six tables with primary keys, foreign keys, `NOT NULL`, `UNIQUE`, `CHECK` and `ENUM` constraints; a `UNIQUE` foreign key models the one-to-zero-or-one relationship between appointments and service records. Sample data of 20 appointments was designed so that every required scenario and every query edge case appears, and fifteen integrity checks confirm that the data is consistent. Ten queries answer the operational questions of the brief, and three views provide appointment details, a summary per service team and a debt report per customer. Particular care is taken to avoid double counting costs when an appointment has several payments. All four scripts run without errors or warnings on MySQL 8.0 under the default strict SQL mode.

The complete project source code, including the SQL scripts, ER diagram and README, is available in the link: https://github.com/hung0302/IST2610-126-PTKD_01_Group-1_Midterm

Contents

List of Figures iv

List of Tables v

1 Introduction and Objectives 1

2 Requirements Analysis and Business Rules 3

3 Database Design 5
3.1 Relationships . . . 5
3.2 Data dictionary . . . 6
3.3 Design decisions . . . 8

4 Schema Implementation (`schema.sql`) 9
4.1 `service_teams`, `stylists` and `customers` . . . 9
4.2 `appointments` . . . 10
4.3 `service_records` . . . 10
4.4 `payments` . . . 10

5 Sample Data and Integrity Verification (`seed.sql`) 11
5.1 Record counts . . . 11
5.2 Scenario coverage . . . 11
5.3 Integrity checks . . . 12
5.4 Constraint violation tests . . . 14

6 Queries (`queries.sql`) 15
6.1 Q1. Completed appointments in January 2025 . . . 15
6.2 Q2. Appointments per team in Q1/2025, including teams with none . . . 16
6.3 Q3. Customers with an appointment that has no service record (excluding Cancelled) . . . 16
6.4 Q4. Average service cost per Completed appointment, by team (all time) . . . 17
6.5 Q5. Top 3 stylists by Completed appointments in 2025 . . . 18
6.6 Q6. Total paid by each customer in 2025 (by `payment_date`) . . . 18
6.7 Q7. Customers with no payment record at all . . . 19
6.8 Q8. All customers ranked by total confirmed service cost . . . 20
6.9 Q9. Cancellation rate per stylist in 2025 . . . 21
6.10 Q10. Customers with more than 3 Completed appointments in 2025 . . . 22

7 Views (`views.sql`) 23
7.1 `v_detail` – one row per appointment . . . 23
7.2 `v_group_summary` – one row per team . . . 25
7.3 `v_debt` – one row per customer . . . 26

8 Conclusion and Recommendations 28
8.1 Conclusion . . . 28
8.2 Lessons Learned . . . 28
8.3 Limitations and Recommendations . . . 29

References 31

Appendices 32

A Repository Structure 32

B Sample Data: the 20 Appointments 33

C Full `schema.sql` 35

List of Figures

3.1 Entity–relationship diagram of `salon_mini` (MySQL Workbench) 6

List of Tables

1.1 Project environment and deliverables 1

2.1 Business rules and how they are enforced 3

3.1 Relationships between the tables 5
3.2 Data dictionary: `service_teams` – Service (specialist) teams 6
3.3 Data dictionary: `stylists` – Hair stylists 6
3.4 Data dictionary: `customers` – Customers 7
3.5 Data dictionary: `appointments` – Appointments (transactions) 7
3.6 Data dictionary: `service_records` – Service records 7
3.7 Data dictionary: `payments` – Payments 7

5.1 Record counts against the minimum required by the brief 11
5.2 Required scenarios and the rows that cover them 12
5.3 Results of the integrity checks in `seed.sql` 13
5.4 Deliberate constraint violations and the response of MySQL 14

6.1 Result of Query 1 15
6.2 Result of Query 2 16
6.3 Result of Query 3 17
6.4 Result of Query 4 18
6.5 Result of Query 5 18
6.6 Result of Query 6 19
6.7 Result of Query 7 20
6.8 Result of Query 8 20
6.9 Result of Query 9 21
6.10 Result of Query 10 22

7.1 Contents of `v_detail` (ID columns of customer, stylist and team omitted for width) 24
7.2 Contents of `v_group_summary` 25
7.3 Contents of `v_debt`, ordered by outstanding debt 27
7.4 Grand totals over `v_debt` 27

B.1 The 20 sample appointments 34

List of Abbreviations

DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ERD	Entity–Relationship Diagram
FK	Foreign Key
PK	Primary Key
Q1–Q10	Queries 1 to 10 of Part C (Chapter 6); “Q1 2025” denotes the first quarter of 2025
SQL	Structured Query Language

Chapter 1

Introduction and Objectives

This report documents Group 1's solution to Assignment 07, *Hair Salon Management*, of the course IST2610 – Database Management (**?**). A hair salon takes bookings from customers, assigns each booking to one stylist, records the service that was performed and collects payment, sometimes in several instalments. The salon needs a database that supports day-to-day record keeping, reports on the activity of each specialist team, and tracking of the money that customers still owe.

The project builds this database, named `salon_mini`, in MySQL (Oracle Corporation, 2026). It exercises the full cycle covered by the course: data definition (DDL), data manipulation (DML), joins, aggregation with `GROUP BY` and `HAVING`, and views. The specific objectives are:

- Design a relational schema with the six tables and columns required by the brief, with primary keys, foreign keys and integrity constraints that keep the data consistent.
- Load sample data that covers every scenario requested by the brief, and verify referential integrity and business rules with queries.
- Write ten independent queries that answer the operational questions in Part C.
- Create three reusable views: appointment details, a summary per service team, and a debt report per customer.
- Deliver four scripts that run in order on a newly created database: `schema.sql` → `seed.sql` → `queries.sql` → `views.sql`.

Table 1.1: Project environment and deliverables

Item	Value
DBMS	MySQL 8.0 (tested on 8.0.46; `CHECK` constraints require 8.0.16 or later)
Tool	MySQL Workbench (scripting and ER diagram)
Database	`salon_mini`
SQL mode	MySQL 8.0 default, including `ONLY_FULL_GROUP_BY` and `STRICT_TRANS_TABLES`
Deliverables	`schema.sql`, `seed.sql`, `queries.sql`, `views.sql` (plus `README.md` and `schema.png`)

Verification. All four scripts were executed in order on a freshly created MySQL 8.0.46 server under the default strict SQL mode. They completed with no errors and no warnings. Every result table in this report is the actual output of that run, not a hand-written expectation.

The rest of the report is organised as follows. Chapter 2 states the business rules, Chapter 3 the database design and Chapter 4 its implementation. Chapter 5 presents the sample data and integrity checks, Chapters 6 and 7 the queries and views, and Chapter **??** concludes.

Chapter 2

Requirements Analysis and Business Rules

The brief describes six kinds of things the salon keeps track of: service teams, stylists, customers, appointments, service records and payments. An appointment is the central business transaction: it links one customer with the stylist who is mainly responsible for it, and it is the anchor for both the service record (what was done and what it cost) and the payments (what has been collected). The service record summarises all work and the total fee of an appointment; individual products and stock are explicitly out of scope.

From the brief we extracted the business rules in Table 2.1. Each rule is mapped to the mechanism that enforces or verifies it; numbers such as #12 refer to the integrity checks in `seed.sql` (Section 5.3).

Table 2.1: Business rules and how they are enforced

ID	Business rule	Enforced / verified by
BR1	A service team has many stylists; each stylist belongs to exactly one team.	FK `stylists.team_id`, `NOT NULL`
BR2	Each appointment belongs to exactly one customer and one main stylist.	FKs `customer_id`, `stylist_id`, both `NOT NULL`
BR3	Valid statuses are Pending, Completed and Cancelled.	`ENUM`, `NOT NULL`
BR4	An appointment has at most one service record.	`UNIQUE` on `service_records.appointment_id`
BR5	Only Completed appointments have a service record, and each Completed appointment has exactly one.	Checks #12 and #21
BR6	Names, business dates and costs are mandatory; cost is never negative.	`NOT NULL`; `CHECK (cost >= 0)`
BR7	A payment is a real collection: `amount > 0`, with a payment date and a method (Cash, BankTransfer, Card).	`CHECK (amount > 0)`, `NOT NULL`, `ENUM`
BR8	Money is collected only after a cost has been confirmed.	Check #13
BR9	Several payments per appointment are allowed, but the total collected never exceeds the confirmed cost. Refunds and prepayments are not handled.	Check #14; `CHECK (amount > 0)` rules out refunds
BR10	Debt = total confirmed cost − total collected. Totals that never occurred are shown as 0, and a cost is never counted twice when an appointment has several payments.	`v_debt` (pre-aggregation and `COALESCE`)
BR11	Yearly reports use 2025; Q1 runs from 01/01/2025 to the end of 31/03/2025. Without a stated period, all data is considered.	Half-open date ranges in queries; views are not time-restricted

Rules BR1–BR4, BR6 and BR7 involve a single table, so they are declared in the schema and MySQL rejects any row that breaks them. Rules BR5, BR8 and BR9 involve several tables or the sum of several rows. The brief forbids `CHECK` constraints that query other tables and makes triggers optional, so these rules are verified with queries in `seed.sql`, as the brief allows.

Chapter 3

Database Design

The schema has six tables, one per entity, using exactly the table and column names required by the brief. `appointments` sits in the middle: it references `customers` and `stylists`, and it is referenced by `service_records` and `payments`. `service_teams` is reached through `stylists`. Every table has a surrogate integer primary key as its first column, following standard relational design practice (Silberschatz et al., 2019; Elmasri and Navathe, 2016).

3.1 Relationships

Table 3.1: Relationships between the tables

Relationship	Cardinality	Implemented by
`service_teams` – `stylists`	1 : N	`stylists.team_id` NOT NULL, FK to `service_teams`
`customers` – `appointments`	1 : N	`appointments.customer_id` NOT NULL, FK to `customers`
`stylists` – `appointments`	1 : N	`appointments.stylist_id` NOT NULL, FK to `stylists`
`appointments` – `service_records`	1 : 0..1	`service_records.appointment_id` NOT NULL **UNIQUE**, FK to `appointments`
`appointments` – `payments`	1 : N	`payments.appointment_id` NOT NULL, FK to `appointments`

The relationship between `appointments` and `service_records` deserves attention. A foreign key alone would allow many records per appointment; the `UNIQUE` constraint on the foreign-key column turns it into a one-to-one relationship. Pending and Cancelled appointments have no record yet, so in practice it is one-to-zero-or-one. Figure 3.1 shows the resulting ER diagram.

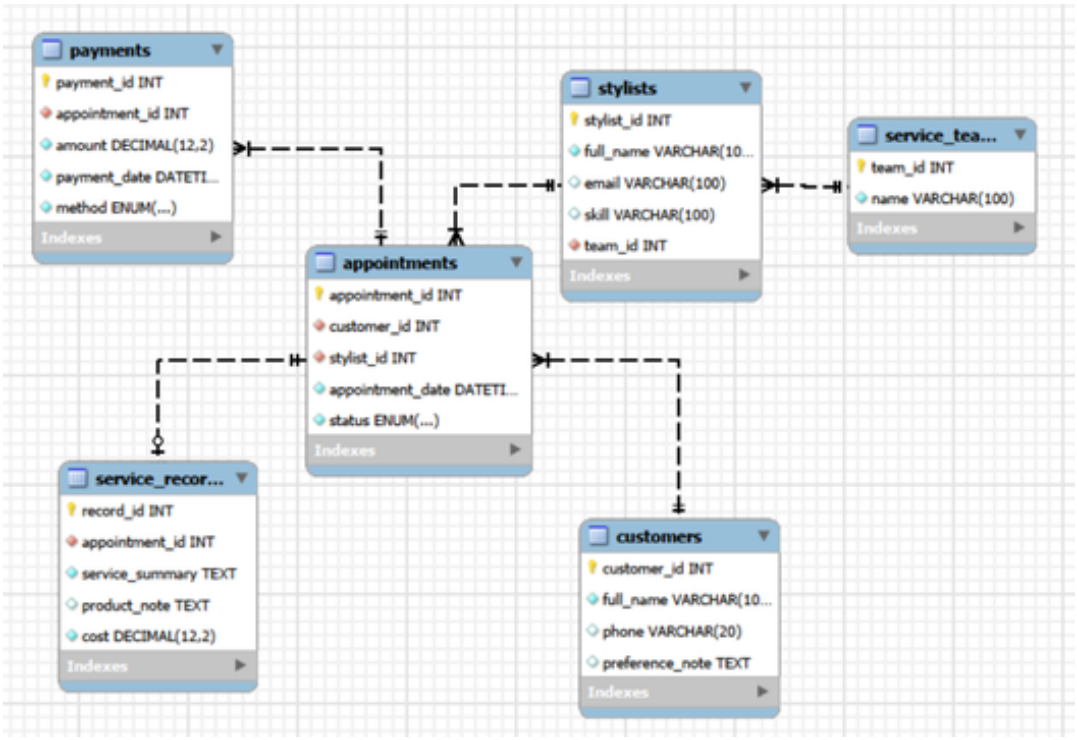


Figure 3.1: Entity–relationship diagram of `salon_mini` (MySQL Workbench)

3.2 Data dictionary

Tables 3.2–3.7 list every column with its type, constraints and meaning.

Table 3.2: Data dictionary: `service_teams` – Service (specialist) teams

Column	Type	Constraints	Description
`team_id`	`INT AUTO_INCREMENT`	PRIMARY KEY	Team identifier
`name`	`VARCHAR(100)`	NOT NULL	Team name

Table 3.3: Data dictionary: `stylists` – Hair stylists

Column	Type	Constraints	Description
`stylist_id`	`INT AUTO_INCREMENT`	PRIMARY KEY	Stylist identifier
`full_name`	`VARCHAR(100)`	NOT NULL	Stylist's full name
`email`	`VARCHAR(100)`	UNIQUE	Contact email; no two stylists share one
`skill`	`VARCHAR(100)`		Main specialist skill
`team_id`	`INT`	NOT NULL, FK	Team the stylist belongs to

Table 3.4: Data dictionary: customers – Customers

Column	Type	Constraints	Description
customer_id	INT AUTO_INCREMENT	PRIMARY KEY	Customer identifier
full_name	VARCHAR(100)	NOT NULL	Customer's full name
phone	VARCHAR(20)		Phone number
preference_note	TEXT		Free-text preferences

Table 3.5: Data dictionary: appointments – Appointments (transactions)

Column	Type	Constraints	Description
appointment_id	INT AUTO_INCREMENT	PRIMARY KEY	Appointment identifier
customer_id	INT	NOT NULL, FK	Customer who booked
stylist_id	INT	NOT NULL, FK	Main stylist in charge
appointment_date	DATETIME	NOT NULL	Business date and time
status	ENUM('Pending', 'Completed', 'Cancelled')	NOT NULL	Current status

Table 3.6: Data dictionary: service_records – Service records

Column	Type	Constraints	Description
record_id	INT AUTO_INCREMENT	PRIMARY KEY	Record identifier
appointment_id	INT	NOT NULL, UNIQUE, FK	Appointment the record belongs to (at most one each)
service_summary	TEXT	NOT NULL	Summary of all work performed
product_note	TEXT		Note on products used
cost	DECIMAL(12,2)	NOT NULL, CHECK (cost $>=$ 0)	Confirmed total fee

Table 3.7: Data dictionary: payments – Payments

Column	Type	Constraints	Description
payment_id	INT AUTO_INCREMENT	PRIMARY KEY	Payment identifier
appointment_id	INT	NOT NULL, FK	Appointment being paid
amount	DECIMAL(12,2)	NOT NULL, CHECK (amount $>$ 0)	Amount actually collected
payment_date	DATETIME	NOT NULL	When the money was collected
method	ENUM('Cash', 'BankTransfer', 'Card')	NOT NULL	Payment method

3.3 Design decisions

1. **Money as `DECIMAL(12,2)`.** Costs and payments are stored as exact decimals, as the brief requires, so sums and differences such as debt are free of floating-point rounding errors.

2. **`DATETIME` for business dates.** A salon books by the hour, so `appointment_date` and `payment_date` keep the time of day as well as the date. All date filters are therefore written as half-open ranges (`>= '2025-01-01' AND < '2025-02-01'`), so that the last day is included up to 23:59:59 and the column is not wrapped in a function, which keeps indexes usable.

3. **`ENUM` for status and method.** Both domains are small and fixed by the brief. An `ENUM` rejects invalid values with no extra lookup table; the trade-off is that adding a value later requires `ALTER TABLE`.

4. **A `UNIQUE` foreign key for the 1 : 0..1 relationship.** Keeping service records in their own table, rather than adding nullable cost columns to `appointments`, follows the table list in the brief and keeps appointments without a result free of empty columns.

5. **Cross-table rules verified by queries.** As explained in Chapter 2, rules that span tables are checked in `seed.sql` rather than with triggers, in line with the brief.

6. **Explicit identifiers in the seed data.** Keys are `AUTO_INCREMENT`, but `seed.sql` inserts explicit IDs so that integrity checks and expected query results are reproducible and easy to cross-reference.

7. **Aggregate before joining.** When costs and payments are combined, each is summed per customer in its own subquery before the join. This prevents a cost from being counted once per payment (Section 7.3).

8. **Conditions in `ON` for outer joins.** Where a report must keep groups with no activity, the date filter is placed in the `ON` clause of the `LEFT JOIN`; placing it in `WHERE` would discard those groups.

Chapter 4

Schema Implementation (schema.sql)

schema.sql creates the salon_mini database and then the six tables in dependency order, parents before children, so that every foreign key refers to a table that already exists: service_teams → stylists → customers → appointments → service_records → payments. Constraints are declared inline with the columns (Oracle Corporation, 2026). The complete script is reproduced in Appendix C; this chapter walks through it table by table.

4.1 service_teams, stylists and customers

These three tables hold reference data. stylists.team_id is NOT NULL and references service_teams, which implements BR1: a stylist cannot exist without a team. email is UNIQUE, so two stylists cannot share a contact address. Only names are mandatory; phone, skill and notes are optional.

```sql
-- 1. Service teams
CREATE TABLE service_teams (
    team_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

-- 2. Stylists
CREATE TABLE stylists (
    stylist_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    skill VARCHAR(100),
    team_id INT NOT NULL,
    FOREIGN KEY (team_id) REFERENCES service_teams(team_id)
);

-- 3. Customers
CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    preference_note TEXT
);
```

Listing 4.1: Reference tables in schema.sql

4.2 appointments

Each appointment must reference an existing customer and stylist (BR2). `status` is an `ENUM` declared `NOT NULL` without a default, so every insert must state the status explicitly (BR3).

```
-- 4. Appointments
CREATE TABLE appointments (
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    stylist_id INT NOT NULL,
    appointment_date DATETIME NOT NULL,
    status ENUM('Pending', 'Completed', 'Cancelled') NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (stylist_id) REFERENCES stylists(stylist_id)
);
```

Listing 4.2: Table appointments

4.3 service_records

`appointment_id` is both a foreign key and `UNIQUE`, which limits each appointment to at most one record (BR4). `cost` is mandatory and protected by `CHECK (cost >= 0)` (BR6). The rule that only Completed appointments have a record cannot be expressed here, because it depends on the status stored in another table; it is verified by check #12.

```
-- 5. Service records
CREATE TABLE service_records (
    record_id INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id INT NOT NULL UNIQUE,
    service_summary TEXT NOT NULL,
    product_note TEXT,
    cost DECIMAL(12,2) NOT NULL,
    CHECK (cost >= 0),
    FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id)
);
```

Listing 4.3: Table service_records

4.4 payments

Every payment refers to an appointment and has a strictly positive amount, a date and one of the three allowed methods (BR7). There is no `UNIQUE` on `appointment_id`, so an appointment can be paid in several instalments. Whether the total stays within the confirmed cost (BR9) is verified by check #14.

```
-- 6. Payments
CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id INT NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    payment_date DATETIME NOT NULL,
    method ENUM('Cash', 'BankTransfer', 'Card') NOT NULL,
    CHECK (amount > 0),
    FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id)
);
```

Listing 4.4: Table payments

Chapter 5

Sample Data and Integrity Verification (`seed.sql`)

`seed.sql` loads the sample data with `INSERT` statements and then runs a series of verification queries. The data is small enough to check by hand but was designed deliberately: every query in Part C and every view in Part D has at least one row that exercises its edge case. The 20 appointments are listed in full in Appendix B.

5.1 Record counts

Table 5.1: Record counts against the minimum required by the brief

Table	Minimum required	Rows loaded	Status
`service_teams`	5	5	Met
`stylists`	6	7	Met
`customers`	10	11	Met
`appointments`	12	20	Met
`service_records`	10	15	Met
`payments`	10	15	Met

The 20 appointments comprise **15 Completed**, **2 Pending** (appointments 15 and 20) and **3 Cancelled** (appointments 14, 16 and 19). Each Completed appointment has exactly one service record, hence 15 rows in `service_records`.

5.2 Scenario coverage

The brief lists situations that the sample data must contain. Table 5.2 shows where each one appears.

Table 5.2: Required scenarios and the rows that cover them

Required scenario	**Covered by**
Transactions in January 2025	Appointments 1, 2, 5 (Completed) and 14 (Cancelled)
Transactions in the other months of Q1 2025	Appointments 3, 6, 7, 13, 15, 18, 19 (February–March 2025)
Transactions outside Q1 but in 2025	Appointments 4, 8, 9, 12, 16, 17, 20 (April–December 2025)
Transactions outside 2025	Appointment 10 (December 2024) and appointment 11 (January 2026)
All three statuses	15 Completed, 2 Pending, 3 Cancelled
A customer with more than 3 Completed transactions in 2025	Customer 1, Nguyen Thi Lan: 4 (appointments 1–4)
A customer with transaction history but no payment	Customer 10, Ngo Van Son: appointment 13 is Completed (cost 200,000) and unpaid; appointment 20 is Pending
A transaction paid in several instalments	Appointment 6: cost 1,100,000, paid 500,000 (Cash) + 600,000 (BankTransfer)
A transaction with an outstanding balance	Appointment 7: cost 950,000, paid 400,000, owes 550,000 (appointment 13 also owes 200,000)
A service team with no transaction in Q1	Team 4 (Hair Care and Treatment) and team 5 (Hair Extensions)
A stylist with no transaction in 2025	Stylist 7, Vu Khanh Linh (only appointment 10, in 2024)
Extra: a customer with no transaction at all	Customer 11, Phan Thi Yen
Extra: payment year differs from appointment year	Appointment 10 paid in 2024; appointment 11 paid in 2026

5.3 Integrity checks

The second half of `seed.sql` contains fifteen verification queries, numbered 7 to 21. They fall into two groups. *Violation finders* search for data that breaks a rule, so the correct result is an empty set. *Scenario confirmations* prove that each situation from Table 5.2 really exists in the data. Table 5.3 lists the actual results.

Table 5.3: Results of the integrity checks in `seed.sql`

#	Check	Expected	Actual result
7	Row counts per table	Meets minimums	5 / 7 / 11 / 20 / 15 / 15
8	Stylists referencing a missing team	0 rows	0 rows
9	Appointments referencing a missing customer or stylist	0 rows	0 rows
10	Service records referencing a missing appointment	0 rows	0 rows
11	Payments referencing a missing appointment	0 rows	0 rows
12	Completed without exactly one record, or non-Completed with a record	0 rows	0 rows
13	Payments for an appointment without a confirmed cost	0 rows	0 rows
14	Total collected exceeds the confirmed cost	0 rows	0 rows
21	Appointments with more than one service record	0 rows	0 rows
15	Customers with more than 3 Completed in 2025	≥ 1 row	Customer 1 (4)
16	Stylists with no appointment in 2025	≥ 1 row	Stylist 7
17	Teams with no appointment in Q1 2025	≥ 1 row	Teams 4 and 5
18	Appointments paid more than once	≥ 1 row	Appointment 6 (2 payments)
19	Appointments with an outstanding balance	≥ 1 row	Appointments 7 and 13
20	Customers with history but no payment	≥ 1 row	Customer 10

Check #12 verifies BR5 in both directions at once by counting the records of every appointment and comparing the count with its status:

```
-- 12. Check appointment status and service records
SELECT a.appointment_id,
       a.status,
       COUNT(sr.record_id) AS record_count
FROM appointments a
LEFT JOIN service_records sr
ON a.appointment_id = sr.appointment_id
GROUP BY a.appointment_id, a.status
HAVING
    (a.status = 'Completed' AND record_count <> 1)
    OR
    (a.status <> 'Completed' AND record_count <> 0);
```

Listing 5.1: Check #12: status versus service record

Check #14 verifies BR9. Because each appointment has at most one service record, joining records to payments and grouping by appointment gives the total collected per appointment without inflating the cost:

```
-- 14. Check total payments against service costs
SELECT sr.appointment_id,
       sr.cost,
       COALESCE(SUM(p.amount), 0) AS total_paid
FROM service_records sr
LEFT JOIN payments p
ON sr.appointment_id = p.appointment_id
GROUP BY sr.appointment_id, sr.cost
HAVING total_paid > sr.cost;
```

Listing 5.2: Check #14: amount collected versus confirmed cost

5.4 Constraint violation tests

Following the brief, statements that deliberately violate a constraint are kept as comments at the end of `seed.sql`, so the file still runs from top to bottom. Each statement was executed separately against the loaded database; MySQL rejected all five with the errors in Table 5.4.

Table 5.4: Deliberate constraint violations and the response of MySQL

Constraint	Test statement	MySQL error
UNIQUE	Second service record for appointment 1	`ERROR 1062`: Duplicate entry '1' for key 'service_records.appointment_id'
CHECK cost	Service record with `cost = -50000`	`ERROR 3819`: Check constraint 'service_records_chk_1' is violated
CHECK amount	Payment with `amount = 0`	`ERROR 3819`: Check constraint 'payments_chk_1' is violated
ENUM	Payment with `method = 'Momo'`	`ERROR 1265`: Data truncated for column 'method'
FOREIGN KEY	Appointment for customer 999	`ERROR 1452`: Cannot add or update a child row: a foreign key constraint fails

```
-- UNIQUE: appointment 1 already has a service record
-- INSERT INTO service_records (appointment_id, service_summary, cost)
-- VALUES (1, 'Duplicate record', 100000);

-- CHECK: cost must be >= 0
-- INSERT INTO service_records (appointment_id, service_summary, cost)
-- VALUES (15, 'Negative cost', -50000);

-- CHECK: amount must be > 0
-- INSERT INTO payments (appointment_id, amount, payment_date, method)
-- VALUES (1, 0, '2025-01-06 10:00:00', 'Cash');

-- ENUM: invalid payment method
-- INSERT INTO payments (appointment_id, amount, payment_date, method)
-- VALUES (1, 10000, '2025-01-06 10:00:00', 'Momo');

-- FOREIGN KEY: customer 999 does not exist
-- INSERT INTO appointments (customer_id, stylist_id, appointment_date,
    status)
-- VALUES (999, 1, '2025-01-01 09:00:00', 'Pending');
```

Listing 5.3: Commented-out violation tests at the end of `seed.sql`

Chapter 6

Queries (queries.sql)

queries.sql contains ten independent queries, numbered as in the brief. Every result includes both the identifier and the name of each object, so that objects with the same name can still be told apart. Time periods use half-open ranges on the DATETIME columns, as explained in Section 3.3.

6.1 Q1. Completed appointments in January 2025

Requirement. List the Completed appointments in January 2025, with the customer, the stylist and the business date appointment_date.

Approach. Inner-join appointments to customers and stylists, keep status Completed and dates from 2025-01-01 up to (but excluding) 2025-02-01, and sort by date.

```sql
-- 1. Completed appointments in January 2025
SELECT a.appointment_id,
       a.appointment_date,
       c.customer_id,
       c.full_name AS customer_name,
       s.stylist_id,
       s.full_name AS stylist_name
FROM appointments a
JOIN customers c ON a.customer_id = c.customer_id
JOIN stylists s ON a.stylist_id = s.stylist_id
WHERE a.status = 'Completed'
  AND a.appointment_date >= '2025-01-01'
  AND a.appointment_date < '2025-02-01'
ORDER BY a.appointment_date;
```

Listing 6.1: Query 1

Table 6.1: Result of Query 1

appointment_id	appointment_date	customer_id	customer_name	stylist_id	stylist_name
1	2025-01-05 09:00	1	Nguyen Thi Lan	1	Nguyen Minh Anh
5	2025-01-12 11:00	2	Tran Van Minh	5	Do Quang Huy
2	2025-01-20 10:00	1	Nguyen Thi Lan	2	Tran Bao Ngoc

Observation. Three appointments qualify. Appointment 14 (25 January) also falls in January but is excluded because it was Cancelled.

6.2 Q2. Appointments per team in Q1/2025, including teams with none

Requirement. Count all appointments per service team in Q1 2025, in any status, and show teams whose count is 0.

Approach. Start from `service_teams` and `LEFT JOIN` to `stylists` and then `appointments`. The Q1 date range sits in the `ON` clause: in `WHERE` it would remove every team without appointments. `COUNT(a.appointment_id)` ignores the NULLs produced by the outer join, so empty teams get 0.

```
-- 2. Total appointments by service team in Q1 2025
SELECT t.team_id,
       t.name AS team_name,
       COUNT(a.appointment_id) AS total_appointments
FROM service_teams t
LEFT JOIN stylists s ON t.team_id = s.team_id
LEFT JOIN appointments a
       ON s.stylist_id = a.stylist_id
      AND a.appointment_date >= '2025-01-01'
      AND a.appointment_date < '2025-04-01'
GROUP BY t.team_id, t.name
ORDER BY t.team_id;
```

Listing 6.2: Query 2

Table 6.2: Result of Query 2

team_id	team_name	total_appointments
1	Hair Cutting and Styling	4
2	Hair Coloring and Bleaching	4
3	Hair Perming and Straightening	3
4	Hair Care and Treatment	0
5	Hair Extensions	0

Observation. Teams 4 and 5 show 0, matching the "team with no transaction in Q1" scenario.

6.3 Q3. Customers with an appointment that has no service record (excluding Cancelled)

Requirement. List customers who have at least one appointment without a service record, ignoring Cancelled appointments. Each customer appears once.

Approach. An anti-join: `LEFT JOIN` appointments to `service_records` and keep rows where `sr.record_id IS NULL`, excluding Cancelled. `DISTINCT` collapses customers with several such appointments into one row.

```sql
-- 3. Customers with appointments without service records,
-- excluding Cancelled appointments
SELECT DISTINCT c.customer_id,
       c.full_name AS customer_name
FROM customers c
JOIN appointments a ON c.customer_id = a.customer_id
LEFT JOIN service_records sr
       ON a.appointment_id = sr.appointment_id
WHERE sr.record_id IS NULL
  AND a.status <> 'Cancelled'
ORDER BY c.customer_id;
```

Listing 6.3: Query 3

Table 6.3: Result of Query 3

customer_id	customer_name
3	Le Thi Hoa
10	Ngo Van Son

Observation. Customer 3 (appointment 15) and customer 10 (appointment 20) each have a Pending appointment, which has no record yet.

6.4 Q4. Average service cost per Completed appointment, by team (all time)

Requirement. Compute the average service fee per Completed appointment for each team over all time, keeping only teams that have incurred costs.

Approach. Inner-join teams → stylists → appointments → service records, keep Completed, and take `AVG(sr.cost)` per team. The inner joins drop teams with no record, satisfying "only teams with costs". Because each appointment has at most one record, no row is duplicated and the average is not distorted.

```sql
-- 4. Average service cost per Completed appointment by team
SELECT t.team_id,
       t.name AS team_name,
       ROUND(AVG(sr.cost), 2) AS average_service_cost
FROM service_teams t
JOIN stylists s ON t.team_id = s.team_id
JOIN appointments a ON s.stylist_id = a.stylist_id
JOIN service_records sr ON a.appointment_id = sr.appointment_id
WHERE a.status = 'Completed'
GROUP BY t.team_id, t.name
ORDER BY t.team_id;
```

Listing 6.4: Query 4

Table 6.4: Result of Query 4

team_id	team_name	average_service_cost
1	Hair Cutting and Styling	226,666.67
2	Hair Coloring and Bleaching	1,250,000
3	Hair Perming and Straightening	1,075,000
4	Hair Care and Treatment	666,666.67
5	Hair Extensions	3,500,000

Observation. All five teams have costs and appear. Team 1 has six records totalling 1,360,000, an average of 226,666.67.

6.5 Q5. Top 3 stylists by Completed appointments in 2025

Requirement. List the three stylists with the most Completed appointments in 2025; break ties by ascending `stylist_id`; include only stylists with transactions.

Approach. Filter Completed appointments in 2025, group by stylist and count. Sort by the count descending and then by `stylist_id` ascending, and take `LIMIT 3`. The inner join keeps only stylists who actually have such appointments.

```
-- 5. Top 3 stylists with most Completed appointments in 2025
SELECT s.stylist_id,
       s.full_name AS stylist_name,
       COUNT(a.appointment_id) AS completed_count
FROM stylists s
JOIN appointments a ON s.stylist_id = a.stylist_id
WHERE a.status = 'Completed'
  AND a.appointment_date >= '2025-01-01'
  AND a.appointment_date < '2026-01-01'
GROUP BY s.stylist_id, s.full_name
ORDER BY completed_count DESC, s.stylist_id ASC
LIMIT 3;
```

Listing 6.5: Query 5

Table 6.5: Result of Query 5

stylist_id	stylist_name	completed_count
1	Nguyen Minh Anh	3
4	Pham Thu Ha	3
5	Do Quang Huy	3

Observation. Stylists 1, 4 and 5 are tied on 3, so the ID tie-break decides their order. Stylists 2 and 3 (2 each) are not in the top three.

6.6 Q6. Total paid by each customer in 2025 (by `payment_date`)

Requirement. Total the amount each customer paid in 2025 according to `payment_date`, showing customers who paid nothing with 0.

Approach. Start from customers and LEFT JOIN to appointments and payments, with the 2025 range on payment_date in the ON clause so every customer is kept. COALESCE(SUM(p.amount), 0) turns a missing total into 0. Costs are not joined, so nothing can be double-counted.

```sql
-- 6. Total payments by customer in 2025
SELECT c.customer_id,
       c.full_name AS customer_name,
       COALESCE(SUM(p.amount), 0) AS total_paid_2025
FROM customers c
LEFT JOIN appointments a ON c.customer_id = a.customer_id
LEFT JOIN payments p
       ON a.appointment_id = p.appointment_id
      AND p.payment_date >= '2025-01-01'
      AND p.payment_date < '2026-01-01'
GROUP BY c.customer_id, c.full_name
ORDER BY c.customer_id;
```

Listing 6.6: Query 6

Table 6.6: Result of Query 6

customer_id	customer_name	total_paid_2025
1	Nguyen Thi Lan	3,200,000
2	Tran Van Minh	180,000
3	Le Thi Hoa	1,100,000
4	Pham Van Nam	400,000
5	Hoang Thi Mai	900,000
6	Do Van Long	480,000
7	Bui Thi Ha	0
8	Vu Van Tuan	0
9	Dang Thi Linh	450,000
10	Ngo Van Son	0
11	Phan Thi Yen	0

Observation. Customers 7 and 8 did pay, but in 2024 and 2026 respectively, so their 2025 total is 0. This shows that the filter is on the payment date, not the appointment date.

6.7 Q7. Customers with no payment record at all

Requirement. List customers with no payment record anywhere in the data, including customers who have never booked.

Approach. NOT EXISTS with a subquery that joins the customer's appointments to payments. A customer with no appointments makes the subquery empty, so they are included automatically.

```sql
-- 7. Customers without any payment records
SELECT c.customer_id,
       c.full_name AS customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM appointments a
```

```
    JOIN payments p ON a.appointment_id = p.appointment_id
    WHERE a.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

Listing 6.7: Query 7

Table 6.7: Result of Query 7

customer_id	customer_name
10	Ngo Van Son
11	Phan Thi Yen

Observation. Customer 10 has bookings but has never paid; customer 11 has never booked.

6.8 Q8. All customers ranked by total confirmed service cost

Requirement. List every customer by total confirmed service cost, descending, regardless of payment; ties are broken by ascending ID.

Approach. `LEFT JOIN` customers to appointments and service records, sum `cost` and use `COALESCE` for customers without costs. Payments are not joined, so costs are not repeated. Sort by total descending, then by `customer_id`.

```
-- 8. Total confirmed service costs by customer
SELECT c.customer_id,
       c.full_name AS customer_name,
       COALESCE(SUM(sr.cost), 0) AS total_service_cost
FROM customers c
LEFT JOIN appointments a ON c.customer_id = a.customer_id
LEFT JOIN service_records sr
       ON a.appointment_id = sr.appointment_id
GROUP BY c.customer_id, c.full_name
ORDER BY total_service_cost DESC, c.customer_id ASC;
```

Listing 6.8: Query 8

Table 6.8: Result of Query 8

customer_id	customer_name	total_service_cost
7	Bui Thi Ha	3,500,000
1	Nguyen Thi Lan	3,200,000
8	Vu Van Tuan	1,800,000
3	Le Thi Hoa	1,100,000
4	Pham Van Nam	950,000
5	Hoang Thi Mai	900,000
6	Do Van Long	480,000
9	Dang Thi Linh	450,000
10	Ngo Van Son	200,000
2	Tran Van Minh	180,000
11	Phan Thi Yen	0

Observation. All eleven customers appear, including customer 11 with 0.

6.9 Q9. Cancellation rate per stylist in 2025

Requirement. For each stylist, Cancelled appointments in 2025 divided by all their 2025 appointments, times 100, rounded to 2 decimals; NULL when the total is 0.

Approach. `LEFT JOIN` stylists to their 2025 appointments (range in `ON`). Cancellations are counted with `SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END)`. The denominator is wrapped in `NULLIF(..., 0)`, so a stylist without appointments yields NULL instead of a division error; multiplying by `100.0` forces decimal division before `ROUND(..., 2)`.

```sql
-- 9. Cancellation rate by stylist in 2025
SELECT s.stylist_id,
       s.full_name AS stylist_name,
       COUNT(a.appointment_id) AS total_appointments,
       SUM(CASE WHEN a.status = 'Cancelled'
                THEN 1 ELSE 0 END) AS cancelled_count,
       ROUND(
           100.0 * SUM(CASE WHEN a.status = 'Cancelled'
                            THEN 1 ELSE 0 END)
           / NULLIF(COUNT(a.appointment_id), 0),
           2
       ) AS cancellation_rate
FROM stylists s
LEFT JOIN appointments a
       ON s.stylist_id = a.stylist_id
      AND a.appointment_date >= '2025-01-01'
      AND a.appointment_date < '2026-01-01'
GROUP BY s.stylist_id, s.full_name
ORDER BY s.stylist_id;
```

Listing 6.9: Query 9

Table 6.9: Result of Query 9

stylist_id	stylist_name	total_appointments	cancelled_count	cancellation_rate
1	Nguyen Minh Anh	3	0	0.00
2	Tran Bao Ngoc	3	1	33.33
3	Le Hoang Nam	4	0	0.00
4	Pham Thu Ha	4	1	25.00
5	Do Quang Huy	3	0	0.00
6	Bui Thanh Mai	1	1	100.00
7	Vu Khanh Linh	0	0	*NULL*

Observation. Stylist 7 had no appointments in 2025, so the rate is NULL. Stylist 6 had one appointment and it was cancelled, giving 100%.

6.10 Q10. Customers with more than 3 Completed appointments in 2025

Requirement. List customers with more than three Completed appointments in 2025, with the count, sorted by the count descending.

Approach. Filter Completed 2025 appointments in `WHERE`, group by customer, and keep groups with `HAVING COUNT(...) > 3`, a condition on the aggregate that `WHERE` cannot express.

```sql
-- 10. Customers with more than 3 Completed appointments in 2025
SELECT c.customer_id,
       c.full_name AS customer_name,
       COUNT(a.appointment_id) AS completed_count
FROM customers c
JOIN appointments a ON c.customer_id = a.customer_id
WHERE a.status = 'Completed'
  AND a.appointment_date >= '2025-01-01'
  AND a.appointment_date < '2026-01-01'
GROUP BY c.customer_id, c.full_name
HAVING COUNT(a.appointment_id) > 3
ORDER BY completed_count DESC;
```

Listing 6.10: Query 10

Table 6.10: Result of Query 10

customer_id	customer_name	completed_count
1	Nguyen Thi Lan	4

Observation. Only customer 1 (Nguyen Thi Lan) qualifies, with 4 Completed appointments.

Chapter 7

Views (`views.sql`)

`views.sql` creates three views. As the brief requires, they cover the whole history and are not tied to 2025, so the salon can use them as standing reports.

7.1 `v_detail` – one row per appointment

Purpose. One row per appointment with its ID, date, status, customer, stylist, service team and service cost.

Design. Appointments are inner-joined to customers, stylists and teams; these foreign keys are all `NOT NULL`, so no appointment is lost. Service records are joined with a `LEFT JOIN`, so appointments without a result are kept with `service_cost` NULL, as required.

```
CREATE VIEW v_detail AS
SELECT
    a.appointment_id,
    a.appointment_date,
    a.status,
    c.customer_id,
    c.full_name AS customer_name,
    s.stylist_id,
    s.full_name AS stylist_name,
    t.team_id,
    t.name AS team_name,
    sr.cost AS service_cost
FROM appointments a
JOIN customers c
    ON a.customer_id = c.customer_id
JOIN stylists s
    ON a.stylist_id = s.stylist_id
JOIN service_teams t
    ON s.team_id = t.team_id
LEFT JOIN service_records sr
    ON a.appointment_id = sr.appointment_id;
```

Listing 7.1: View `v_detail`

Table 7.1: Contents of `v_detail` (ID columns of customer, stylist and team omitted for width)

appointment_id	date	status	customer_ name	stylist_ name	team_name	service_cost
1	2025-01-05 09:00	Completed	Nguyen Thi Lan	Nguyen Minh Anh	Hair Cutting and Styling	250,000
2	2025-01-20 10:00	Completed	Nguyen Thi Lan	Tran Bao Ngoc	Hair Coloring and Bleaching	850,000
3	2025-02-10 14:00	Completed	Nguyen Thi Lan	Le Hoang Nam	Hair Perming and Straighten- ing	1,200,000
4	2025-04-15 09:00	Completed	Nguyen Thi Lan	Pham Thu Ha	Hair Care and Treatment	900,000
5	2025-01-12 11:00	Completed	Tran Van Minh	Do Quang Huy	Hair Cutting and Styling	180,000
6	2025-02-18 13:00	Completed	Le Thi Hoa	Tran Bao Ngoc	Hair Coloring and Bleaching	1,100,000
7	2025-03-08 15:00	Completed	Pham Van Nam	Le Hoang Nam	Hair Perming and Straighten- ing	950,000
8	2025-04-10 10:00	Completed	Hoang Thi Mai	Pham Thu Ha	Hair Care and Treatment	650,000
9	2025-05-12 09:00	Completed	Do Van Long	Nguyen Minh Anh	Hair Cutting and Styling	300,000
10	2024-12-15 14:00	Completed	Bui Thi Ha	Vu Khanh Linh	Hair Extensions	3,500,000
11	2026-01-10 10:00	Completed	Vu Van Tuan	Bui Thanh Mai	Hair Coloring and Bleaching	1,800,000
12	2025-07-20 13:00	Completed	Dang Thi Linh	Pham Thu Ha	Hair Care and Treatment	450,000
13	2025-03-15 09:00	Completed	Ngo Van Son	Do Quang Huy	Hair Cutting and Styling	200,000
14	2025-01-25 10:00	Cancelled	Tran Van Minh	Tran Bao Ngoc	Hair Coloring and Bleaching	*NULL*
15	2025-02-25 14:00	Pending	Le Thi Hoa	Le Hoang Nam	Hair Perming and Straighten- ing	*NULL*
16	2025-10-05 11:00	Cancelled	Pham Van Nam	Pham Thu Ha	Hair Care and Treatment	*NULL*
17	2025-12-12 09:00	Completed	Hoang Thi Mai	Nguyen Minh Anh	Hair Cutting and Styling	250,000
18	2025-03-22 10:00	Completed	Do Van Long	Do Quang Huy	Hair Cutting and Styling	180,000
19	2025-03-28 13:00	Cancelled	Vu Van Tuan	Bui Thanh Mai	Hair Coloring and Bleaching	*NULL*
20	2025-06-15 10:00	Pending	Ngo Van Son	Le Hoang Nam	Hair Perming and Straighten- ing	*NULL*

Observation. The view has 20 rows, one per appointment. The five Pending or Cancelled

appointments (14, 15, 16, 19, 20) are kept with a NULL cost.

7.2 v_group_summary – one row per team

Purpose. One row per service team with its ID, name, total transactions, number Completed, number Cancelled and total service cost, keeping teams with no transactions.

Design. A chain of LEFT JOINs starting from service_teams, so no team can disappear. Completed and Cancelled counts use SUM(CASE ...), and COALESCE turns a missing cost total into 0. Each appointment has at most one record, so joining service_records does not duplicate appointment rows and both COUNT and SUM stay correct.

```sql
CREATE VIEW v_group_summary AS
SELECT
    t.team_id,
    t.name AS team_name,
    COUNT(a.appointment_id) AS total_appointments,
    SUM(CASE
        WHEN a.status = 'Completed' THEN 1
        ELSE 0
    END) AS completed_count,
    SUM(CASE
        WHEN a.status = 'Cancelled' THEN 1
        ELSE 0
    END) AS cancelled_count,
    COALESCE(SUM(sr.cost), 0) AS total_service_cost
FROM service_teams t
LEFT JOIN stylists s
    ON t.team_id = s.team_id
LEFT JOIN appointments a
    ON s.stylist_id = a.stylist_id
LEFT JOIN service_records sr
    ON a.appointment_id = sr.appointment_id
GROUP BY t.team_id, t.name;
```

Listing 7.2: View v_group_summary

Table 7.2: Contents of v_group_summary

team_id	team_name	total_ appointments	completed_ count	cancelled_ count	total_ service_cost
1	Hair Cutting and Styling	6	6	0	1,360,000
2	Hair Coloring and Bleaching	5	3	2	3,750,000
3	Hair Perming and Straightening	4	2	0	2,150,000
4	Hair Care and Treatment	4	3	1	2,000,000
5	Hair Extensions	1	1	0	3,500,000

Observation. The total_appointments column adds up to 20 and the team costs add up to 12,760,000, the same grand total as v_debt below. This is a useful cross-check between two independently written views.

7.3 v_debt – one row per customer

Purpose. One row per customer with ID, name, total confirmed cost, total paid and outstanding debt, keeping customers with no cost or no payment.

Design. This is the view most exposed to double counting. Joining service_records directly to payments and then summing would repeat a cost once for every payment of the same appointment. In our data, appointment 6 (cost 1,100,000) has two payments; a naïve join reports customer 3's cost as 2,200,000 and the grand total as 13,860,000 instead of 12,760,000. The view therefore aggregates costs (subquery sc) and payments (subquery pm) per customer *separately*, and only then LEFT JOINs both onto customers. Debt is cost minus payments, with COALESCE turning totals that never occurred into 0.

```
CREATE VIEW v_debt AS
SELECT
    c.customer_id,
    c.full_name AS customer_name,
    COALESCE(sc.total_service_cost, 0)
        AS total_service_cost,
    COALESCE(pm.total_paid, 0)
        AS total_paid,
    COALESCE(sc.total_service_cost, 0)
        - COALESCE(pm.total_paid, 0)
        AS outstanding_debt
FROM customers c

LEFT JOIN (
    SELECT
        a.customer_id,
        SUM(sr.cost) AS total_service_cost
    FROM appointments a
    JOIN service_records sr
        ON a.appointment_id = sr.appointment_id
    GROUP BY a.customer_id
) sc
    ON c.customer_id = sc.customer_id

LEFT JOIN (
    SELECT
        a.customer_id,
        SUM(p.amount) AS total_paid
    FROM appointments a
    JOIN payments p
        ON a.appointment_id = p.appointment_id
    GROUP BY a.customer_id
) pm
    ON c.customer_id = pm.customer_id;
```

Listing 7.3: View v_debt

Table 7.3: Contents of v_debt, ordered by outstanding debt

customer_id	customer_name	total_service_cost	total_paid	outstanding_debt
4	Pham Van Nam	950,000	400,000	550,000
10	Ngo Van Son	200,000	0	200,000
1	Nguyen Thi Lan	3,200,000	3,200,000	0
2	Tran Van Minh	180,000	180,000	0
3	Le Thi Hoa	1,100,000	1,100,000	0
5	Hoang Thi Mai	900,000	900,000	0
6	Do Van Long	480,000	480,000	0
7	Bui Thi Ha	3,500,000	3,500,000	0
8	Vu Van Tuan	1,800,000	1,800,000	0
9	Dang Thi Linh	450,000	450,000	0
11	Phan Thi Yen	0	0	0

Table 7.4: Grand totals over v_debt

total_service_cost	total_paid	total_outstanding_debt
12,760,000	12,010,000	750,000

Observation. Confirmed costs total 12,760,000, of which 12,010,000 has been collected, leaving 750,000 outstanding. The debt comes from two customers: customer 4 owes 550,000 after a partial payment, and customer 10 owes 200,000 for a completed but unpaid service. Customer 3, whose appointment was paid in two instalments, shows a cost equal to the amount paid, which confirms that the cost was not counted twice. Customer 11, with no activity, still appears with zeros.

Chapter 8

Conclusion and Recommendations

8.1 Conclusion

This project successfully designed and implemented `salon_mini`, a relational database for managing the core operations of a hair salon. The project covered the four main requirements of Assignment 07, including database design, sample data implementation and integrity verification, SQL query development, and view creation.

The database consists of six interconnected tables: `service_teams`, `stylists`, `customers`, `appointments`, `service_records`, and `payments`. Primary keys, foreign keys, UNIQUE constraints, CHECK constraints, NOT NULL constraints, and ENUM data types were implemented to maintain data consistency and enforce the defined business rules. The sample dataset includes 20 appointments and satisfies the required record counts and business scenarios. In addition, fifteen integrity checks were developed to verify relationships between tables, appointment statuses, service records, and payment amounts.

Ten SQL queries were implemented to retrieve operational information, including appointment schedules, team performance, stylist statistics, customer activity, and payment history. Three views were also created to provide consolidated information about appointments, service teams, and customer outstanding debts.

One important aspect of the project was ensuring the accuracy of financial calculations. By aggregating service costs and payments separately before joining them in `v_debt`, the database avoids double counting when an appointment has multiple payments. Based on the sample data, the total confirmed service cost is 12,760,000, while the total amount collected is 12,010,000, resulting in an outstanding debt of 750,000.

Overall, the project demonstrates how relational database design, integrity constraints, SQL queries, and views can work together to support the daily management and financial tracking of a hair salon.

8.2 Lessons Learned

Throughout the project, several important lessons were learned about relational database design, SQL implementation, and data integrity.

- **The importance of proper database design.** Defining appropriate relationships, primary keys, and foreign keys is essential for organizing data and preventing inconsistencies. Separating appointments, service records, and payments into different tables also allows the database to represent real business operations more accurately.

- **The role of constraints in maintaining data integrity.** Constraints such as NOT NULL, UNIQUE, CHECK, and ENUM help prevent invalid data from entering the database. However, rules involving multiple tables may require additional verification queries or database triggers because they cannot always be enforced through ordinary table constraints.
- **The importance of understanding JOIN operations.** The project demonstrated that the placement of conditions in JOIN operations matters. In particular, conditions placed in the WHERE clause can unintentionally eliminate rows preserved by a LEFT JOIN. Therefore, filtering conditions should be carefully positioned when records with no related activity must still appear in the results.
- **Avoiding double counting in aggregate queries.** Joining multiple one-to-many relationships can multiply rows and produce incorrect aggregate results. The implementation of `v_debt` demonstrated the importance of aggregating related data separately before joining the results.
- **The value of designing test data around business scenarios.** Sample data should not only satisfy minimum record counts but also cover important edge cases. Examples include multiple payments for one appointment, unpaid completed appointments, customers without transaction history, and stylists without appointments during a specific period. These scenarios help verify whether queries and views work correctly under different conditions.

8.3 Limitations and Recommendations

Although the `salon_mini` database satisfies the requirements of the assignment, several limitations remain. The following points identify the current limitations and propose corresponding improvements for future development.

1. **Cross-table Integrity Enforcement**

   Limitation: Some business rules involving multiple tables, such as requiring a service record for every Completed appointment and ensuring that total payments do not exceed the confirmed service cost, are currently verified through separate SQL queries. These rules are not automatically enforced when new records are inserted or updated.

   Recommendation: Implement database triggers to validate these conditions whenever relevant records are inserted or updated. This would help prevent inconsistent data and improve the reliability of the database.

2. **Limited Service Management**

   Limitation: The current `service_records` table stores only a general description of the services performed and their total cost. It does not provide detailed information about individual services included in an appointment.

   Recommendation: Introduce a service catalogue and a service detail table to manage individual service types, their prices, and the services performed during each appointment. This would support more detailed revenue analysis and service performance evaluation.

3. **Lack of Product and Inventory Management**

   Limitation: The current database does not include dedicated tables for managing salon products or inventory. Information about products used during services is stored only

as optional notes in `service_records`, making it difficult to track product usage and stock levels.

Recommendation: Extend the database with product and inventory tables to manage product information, monitor stock quantities, and record products used during services. This would improve inventory control and support daily salon operations.

4. **Query Performance and Scalability**

   Limitation: The current database is designed and tested using a relatively small sample dataset. As the volume of appointments and payment records increases, queries involving date ranges, appointment statuses, and payment dates may require more processing time.

   Recommendation: Introduce appropriate indexes on frequently queried columns, such as `appointment_date`, `status`, and `payment_date`. Query performance should also be monitored as the database grows to identify further optimization opportunities.

Overall, addressing these limitations would improve the database's data integrity, operational capabilities, and performance. These recommendations provide a foundation for developing `salon_mini` into a more comprehensive solution for hair salon management.

References

Elmasri, R. and Navathe, S. B. (2016), *Fundamentals of Database Systems*, 7th edn, Pearson, Boston.

Oracle Corporation (2026), *MySQL 8.0 Reference Manual*. Sections on CHECK constraints, FOREIGN KEY constraints and CREATE VIEW. Accessed November 2026.
URL: *https://dev.mysql.com/doc/refman/8.0/en/*

Silberschatz, A., Korth, H. F. and Sudarshan, S. (2019), *Database System Concepts*, 7th edn, McGraw-Hill, New York.

Appendix A

Repository Structure

The project repository contains the four required scripts, the ER diagram and a README:

```
IST2610-126-PTKD_01_Group-1_Midterm/
|-- schema.sql      -- Part A: database, tables, constraints
|-- seed.sql        -- Part B: sample data, integrity checks, violation
    tests
|-- queries.sql     -- Part C: ten queries
|-- views.sql       -- Part D: three views and view checks
|-- schema.png      -- ER diagram (Figure 3.1)
`-- README.md       -- overview and run instructions
```

To run the project from the command line (the first line is only needed when re-running on a server where `salon_mini` already exists, because `schema.sql` uses `CREATE DATABASE`):

```
mysql -u root -p -e "DROP DATABASE IF EXISTS salon_mini;"
mysql -u root -p < schema.sql
mysql -u root -p < seed.sql
mysql -u root -p < queries.sql
mysql -u root -p < views.sql
```

In MySQL Workbench, open each script with *File* → *Open SQL Script* and execute it in full, in the same order.

Appendix B

Sample Data: the 20 Appointments

Table B.1 lists every appointment with its confirmed cost and the total actually paid. Team numbers refer to: 1 Hair Cutting and Styling, 2 Hair Coloring and Bleaching, 3 Hair Perming and Straightening, 4 Hair Care and Treatment, 5 Hair Extensions.

Table B.1: The 20 sample appointments

ID	Date	Status	Customer	Stylist	Team	Cost	Paid	#Pay
1	2025-01-05 09:00	Completed	1 - Nguyen Thi Lan	1 - Nguyen Minh Anh	1	250,000	250,000	1
2	2025-01-20 10:00	Completed	1 - Nguyen Thi Lan	2 - Tran Bao Ngoc	2	850,000	850,000	1
3	2025-02-10 14:00	Completed	1 - Nguyen Thi Lan	3 - Le Hoang Nam	3	1,200,000	1,200,000	1
4	2025-04-15 09:00	Completed	1 - Nguyen Thi Lan	4 - Pham Thu Ha	4	900,000	900,000	1
5	2025-01-12 11:00	Completed	2 - Tran Van Minh	5 - Do Quang Huy	1	180,000	180,000	1
6	2025-02-18 13:00	Completed	3 - Le Thi Hoa	2 - Tran Bao Ngoc	2	1,100,000	1,100,000	2
7	2025-03-08 15:00	Completed	4 - Pham Van Nam	3 - Le Hoang Nam	3	950,000	400,000	1
8	2025-04-10 10:00	Completed	5 - Hoang Thi Mai	4 - Pham Thu Ha	4	650,000	650,000	1
9	2025-05-12 09:00	Completed	6 - Do Van Long	1 - Nguyen Minh Anh	1	300,000	300,000	1
10	2024-12-15 14:00	Completed	7 - Bui Thi Ha	7 - Vu Khanh Linh	5	3,500,000	3,500,000	1
11	2026-01-10 10:00	Completed	8 - Vu Van Tuan	6 - Bui Thanh Mai	2	1,800,000	1,800,000	1
12	2025-07-20 13:00	Completed	9 - Dang Thi Linh	4 - Pham Thu Ha	4	450,000	450,000	1
13	2025-03-15 09:00	Completed	10 - Ngo Van Son	5 - Do Quang Huy	1	200,000	0	0
14	2025-01-25 10:00	Cancelled	2 - Tran Van Minh	2 - Tran Bao Ngoc	2	–	–	0
15	2025-02-25 14:00	Pending	3 - Le Thi Hoa	3 - Le Hoang Nam	3	–	–	0
16	2025-10-05 11:00	Cancelled	4 - Pham Van Nam	4 - Pham Thu Ha	4	–	–	0
17	2025-12-12 09:00	Completed	5 - Hoang Thi Mai	1 - Nguyen Minh Anh	1	250,000	250,000	1
18	2025-03-22 10:00	Completed	6 - Do Van Long	5 - Do Quang Huy	1	180,000	180,000	1
19	2025-03-28 13:00	Cancelled	8 - Vu Van Tuan	6 - Bui Thanh Mai	2	–	–	0
20	2025-06-15 10:00	Pending	10 - Ngo Van Son	3 - Le Hoang Nam	3	–	–	0

Appendix C

Full `schema.sql`

The listing below is read directly from `sql/schema.sql` in this Overleaf project, so it always matches the submitted script.

```sql
CREATE DATABASE salon_mini;
USE salon_mini;

-- 1. Service teams
CREATE TABLE service_teams (
    team_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

-- 2. Stylists
CREATE TABLE stylists (
    stylist_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    skill VARCHAR(100),
    team_id INT NOT NULL,
    FOREIGN KEY (team_id) REFERENCES service_teams(team_id)
);

-- 3. Customers
CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    preference_note TEXT
);

-- 4. Appointments
CREATE TABLE appointments (
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    stylist_id INT NOT NULL,
    appointment_date DATETIME NOT NULL,
    status ENUM('Pending', 'Completed', 'Cancelled') NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (stylist_id) REFERENCES stylists(stylist_id)
);

-- 5. Service records
CREATE TABLE service_records (
    record_id INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id INT NOT NULL UNIQUE,
```

```
    service_summary TEXT NOT NULL,
    product_note TEXT,
    cost DECIMAL(12,2) NOT NULL,
    CHECK (cost >= 0),
    FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id)
);

-- 6. Payments
CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id INT NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    payment_date DATETIME NOT NULL,
    method ENUM('Cash', 'BankTransfer', 'Card') NOT NULL,
    CHECK (amount > 0),
    FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id)
);
```

Listing C.1: Complete schema.sql